IIR-Robust-Retrieval

Tài liệu onboarding và kế hoạch triển khai baseline GeM (B0)

Phiên bản	1.1 · 08/10/2026
Phạm vi	B0 reproduction · C0–C4 · Data · Kaggle · GitHub collaboration
Trạng thái	C0 đã commit; Kaggle runtime, dataset và B0 mAP chưa được xác nhận

Tóm tắt

Tài liệu dành cho cả ba thành viên, bắt đầu từ cách hoạt động của Instance-Level Image Retrieval và GeM, tiếp đến tiến độ B0, nhiệm vụ mỗi người, kế hoạch triển khai C1–C4, quy trình GitHub và hướng đánh giá robustness. Đây là kế hoạch kỹ thuật nội bộ, không phải báo cáo kết quả thí nghiệm. Mọi tuyên bố về chạy thành công cần dựa trên log và kết quả thực tế.

0. Tổng quan: Chúng ta đang làm gì?

Instance-Level Image Retrieval (IIR) là bài toán: cho một ảnh truy vấn (query), tìm và xếp hạng các ảnh trong kho (gallery) chứa cùng một đối tượng hoặc địa điểm cụ thể. Ví dụ: hai ảnh chụp cùng một tòa nhà từ góc khác nhau phải được nhận diện là liên quan, trong khi tòa nhà khác không được xếp quá cao.

Pipeline cơ bản: Ảnh → CNN trích xuất feature map → GeM pooling tạo vector toàn cục → L2 normalization → so sánh vector query–gallery → ranking → tính mAP.

Baseline B0 tái lập phương pháp trong bài “Fine-tuning CNN Image Retrieval with No Human Annotation” (Radenović, Tolias, Chum), với source tham khảo chính thức: https://github.com/filipradenovic/cnnimageretrieval-pytorch. Mục tiêu B0 là chạy đúng, có thể lặp lại, đo được kết quả, và hiểu rõ những sai lệch so với paper/source. Không tự ý thay loss, backbone hay mining để “tăng điểm” trong giai đoạn compatibility.

Sau B0, nhóm đánh giá general robustness: hiệu năng retrieval suy giảm thế nào khi ảnh thay đổi về blur, noise, ánh sáng, che khuất, clutter, scale và góc nhìn thực tế. Dựa trên bằng chứng, nhóm mới chọn B1. B1 chưa phải tên của một mô hình đã chốt.

1. Hiện tại nhóm đang ở đâu?

Hạng mục	Trạng thái đã biết	Ý nghĩa / việc còn thiếu
Source GeM gốc / main	Đã import, commit 886c117	Giữ bản tham chiếu ổn định, không tùy tiện sửa.
B0 structure	Đã commit f00e6b2	Có thư mục configs, report, scripts, notebooks, experiments.

Hạng mục	Trạng thái đã biết	Ý nghĩa / việc còn thiếu
C0 modern API	Đã commit 4245b79	Thay API Pillow/NumPy và viết preflight; chưa kiểm thử runtime Kaggle.
GitHub remote	Đã cấu hình origin FUAVX	Chưa có bằng chứng push thành công trong output đã cung cấp.
C1 — Paths / data / device	Chưa triển khai	Chưa xác minh Kaggle paths, offline data, GPU.
C2 — Checkpoint / resume	Chưa triển khai	Chưa có kiểm thử lưu và tiếp tục training.
C3 — Training stability	Chưa triển khai	Chưa có smoke test loss / sampling / gradients.
C4 — Evaluation validation	Chưa triển khai	Chưa kiểm tra pipeline query → ranking → AP/mAP.
Dataset	Chưa xác nhận đủ 3 bộ	Hoàng cần lập manifest và xác nhận dữ liệu thực có.
B0 training / mAP	Chưa có kết quả được xác nhận	Không được tuyên bố baseline đã reproduce.
B1 robustness	Chưa chốt phương pháp	Chỉ xây giao thức đánh giá và tìm gap sau B0.

1.1 C0 cụ thể đã làm gì?

- cirtorch/datasets/datahelpers.py: Image.ANTIALIAS → Image.Resampling.LANCZOS; giữ nguyên thumbnail và kích thước.
- cirtorch/utils/evaluate.py: np.in1d → np.isin cho positive và junk membership; không đổi công thức AP/mAP.
- configs/B0/README.md: hướng dẫn dùng môi trường hiện đại và script preflight; không cài các dependency pin từ thời PyTorch 1.0.

Các kiểm tra đã báo cáo: syntax parsing, git diff --check; chưa import/runtime vì môi trường local thiếu NumPy/Pillow/torch/torchvision. Vì vậy “C0 đã commit” khác với “C0 đã chạy tốt trên Kaggle”.

1.2 C0–C4 là gì? Vì sao nhóm cần làm?

C0–C4 là tên các giai đoạn kỹ thuật do nhóm tự đặt để hiện đại hóa và kiểm chứng code GeM trước khi chạy B0 trên Kaggle. Đây **không phải** các bước trong bài báo gốc và cũng **không phải** năm phương pháp cải tiến robustness. Chúng giải quyết vấn đề kỹ thuật: source được viết cho thư viện cũ, đường dẫn local, vòng đời phiên GPU và quy trình đánh giá cần được xác nhận lại.

Giai đoạn	Câu hỏi cần trả lời	Đầu ra tối thiểu	Phụ trách chính
C0 — API Compatibility	API Python/Pillow/NumPy cũ có còn chạy?	Bản vá tương thích, preflight, không đổi thuật toán	Phương; Nguyên review
C1 — Paths, Data & Device	Code có đọc đúng dữ liệu, weights, GPU và ghi đúng nơi trên Kaggle?	Config đường dẫn; load được ảnh và weights; xác nhận thiết bị	Phương + Hoàng

Giai đoạn	Câu hỏi cần trả lời	Đầu ra tối thiểu	Phụ trách chính
C2 — Checkpoint & Resume	Kaggle ngắt giữa chừng thì có tiếp tục train đúng được không?	Save/load checkpoint, LR/ optimizer khớp policy	Phương; Nguyên review scheduler
C3 — Training Stability	Một vòng train thật có lấy mẫu đúng và cập nhật weights ổn định?	Smoke test loss/gradients/ sampling, log lỗi rõ	Phương + Nguyên
C4 — Evaluation Validation	mAP báo cáo có được tính đúng protocol?	Kiểm tra crop, resize, ranks, positive/junk, AP/mAP	Nguyên + Hoàng; Phương tích hợp

Phân biệt ba mốc: (i) *đã sửa code* không đồng nghĩa (ii) *đã smoke-test trên Kaggle*, và (ii) chưa đồng nghĩa (iii) *đã reproduce được mAP của paper*. C0 hiện mới chắc chắn ở mốc (i). Sau C4, nhóm vẫn phải huấn luyện B0, đánh giá và ghi rõ khác biệt cấu hình so với paper.

1.3 Những khái niệm quan trọng

Thuật ngữ	Giải thích dễ hiểu
B0 / baseline	Mốc đối chứng để biết cải tiến B1 có thật sự tốt hơn không.
Reproduction	Chạy lại phương pháp với cấu hình và điều kiện ghi chép minh bạch.
GeM	Generalized Mean Pooling: gộp feature map thành một vector mô tả ảnh; p là tham số pooling có thể học.
Descriptor	Vector đại diện cho ảnh; hai ảnh cùng instance lý tưởng có vector gần nhau.
SfM	Structure-from-Motion: nguồn thông tin hình học/cluster dùng tạo cặp ảnh liên quan, không phải model đang train.
Positive / negative	Ảnh cùng instance / khác instance dùng để học khoảng cách descriptor.
Hard-negative mining	Tìm các negative dễ bị nhầm để quá trình học có tín hiệu khó hơn.
mAP	Mean Average Precision: đánh giá thứ tự ranking so với ground truth; không phải accuracy classification.
Checkpoint	Trạng thái lưu để tiếp tục train và tái lập kết quả.
Commit / branch / PR	Ảnh chụp thay đổi code / nhánh công việc / yêu cầu review trước khi gộp.

2. Phân công nhóm và cách phối hợp

Nguyên tắc: chia trách nhiệm đầu ra, không chia kiến thức. Cả ba phải hiểu từ dữ liệu → model → training → evaluation. Người phụ trách chịu trách nhiệm tiến độ, kiểm tra và giải thích, không có nghĩa là tự làm một mình.

Người	Vai trò	Đầu ra vòng đầu
Phương	Training & Integration	Kaggle preflight; C1 data/device; C2 resume; cùng Nguyên smoke test C3; tích hợp C4 và B0 run log.

Người	Vai trò	Đầu ra vòng đầu
Nguyên	Methodology & Model Analysis	Bảng đối chiếu paper/source; kiểm tra scheduler C2, loss/mining C3 và tính đúng mAP C4.
Hoàng	Data & Evaluation Preparation	Manifest SfM/Oxford/Paris; Kaggle paths C1; kiểm tra metadata, query bbox và ground truth C4.

2.1 Phương — Training & Integration

- Xác nhận GitHub remote, trạng thái push, tạo notebook Kaggle GPU và ghi phiên bản môi trường.
- Phối hợp Hoàng xác nhận dữ liệu được mount dưới /kaggle/input; không viết cứng đường dẫn khi chưa biết layout.
- Điều phối Codex sửa C1–C4 từng patch nhỏ, review diff, không đổi thuật toán ngầm.
- Thiết lập output dưới /kaggle/working; checkpoint định kỳ, resume và log cấu hình/seed.
- Chạy smoke test ngắn trước full B0; lưu metric, lỗi, checkpoint và hash commit.

2.2 Nguyên — Methodology & Analysis

- Đọc paper và code liên quan VGG16, GeM, L2, contrastive loss, tuple sampling và mining.
- Viết bảng: paper nói gì / source làm gì / ảnh hưởng / đề xuất giữ hay thay / quyết định nhóm.
- Đặc biệt kiểm tra scheduler, hard-negative refresh, margin, optimizer, validation và cách chọn best checkpoint.
- Không tự sửa model, pooling hoặc loss trong C1–C4. Nếu phát hiện mismatch, mở issue và trao đổi trước.
- Chuẩn bị mẫu phân tích các query sai sau khi B0 có kết quả.

2.3 Hoàng — Data & Evaluation

- Xác minh retrieval-SfM-120k, Oxford5k, Paris6k: nguồn, tình trạng tải, dung lượng, file thực tế, metadata và ground truth.
- Kiểm tra cids/cluster/qidxs/pidxs trong train/val SfM; kiểm tra query bbox, positive/junk ground truth của benchmark.
- Đề xuất cách mount Kaggle dataset chỉ đọc và mapping path; không upload ảnh/checkpoint lên Git.
- Không random split ảnh SfM gây rò rỉ cluster; nếu giảm dữ liệu, lấy theo toàn cluster và ghi lại quy tắc.
- Tạo manifest, báo thiếu/hỏng dữ liệu và test một số ảnh/ID có load được.

2.4 Cách nghiệm thu và giao tiếp

Mỗi deliverable cần có: (1) mục tiêu; (2) file/commit hoặc notebook; (3) những gì đã xác minh; (4) điều chưa chắc chắn; (5) câu hỏi cần nhóm quyết định. Gửi cập nhật ngắn khi hoàn tất, không chỉ nhắn “xong rồi”.

3. Quy trình triển khai kỹ thuật: C0–C4

3.1 Bước 0: Xác minh remote và commit

```
cd ~/projects/IIR-Robust-Retrieval
git status
git log --oneline -4
git remote -v
git branch -vv
```

Nếu chưa push: git push -u origin main; git push -u origin B0. Chỉ xem là thành công khi terminal xác nhận hoặc thấy hai branch trên GitHub.

3.2 Kaggle preflight (chưa cần dữ liệu)

```
import sys
import torch
import torchvision
import numpy as np
import PIL

print(sys.version)
print(torch.__version__, torchvision.__version__)
print(np.__version__, PIL.__version__)
print(torch.cuda.is_available(), torch.version.cuda)
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

Ghi output vào run record; nếu import lỗi, dừng để kiểm tra cặp torch/torchvision. Không cài requirements.txt lịch sử (torch==1.0.0, torchvision==0.2.1) lên Kaggle hiện đại. Preflight này chưa kiểm tra import toàn bộ cirtorch.

3.3 C1 — Kaggle Paths, Offline Data & Device

C1 giải quyết vấn đề gì? Source GeM tham chiếu thường tìm dữ liệu dưới thư mục `data/` cạnh code, có logic tải dữ liệu về máy, và một số đoạn dùng CUDA trực tiếp. Kaggle tách nơi chứa input (thường chỉ đọc) và nơi ghi output; nếu giữ nguyên giả định local, code có thể không tìm thấy ảnh, cố download lại dữ liệu, ghi nhầm vào thư mục chỉ đọc hoặc gọi `.cuda()` sai thiết bị.

Các từ cần hiểu: *data path* là đường dẫn bộ ảnh và metadata; *pretrained weights* là bộ tham số VGG16 Caffe gốc cần nạp, **không thay bằng weights ImageNet mặc định của torchvision**; *device* là GPU/CPU nơi tensor và model chạy; *offline data* là bộ dữ liệu đã được mount trước, không download trong training.

Việc cần làm theo thứ tự:

1. **Hoàng xác nhận dữ liệu thật:** bộ `retrieval-SfM-120k` có train/val metadata, các `cids`, `cluster`, `qidxs`, `pidxs` và ảnh CID; Oxford5k/Paris6k có ảnh, ground-truth, query bounding boxes. Lập manifest ghi nguồn, số file, kích thước, đường dẫn Kaggle chính xác và kiểm tra mở được mẫu ảnh. Hiện cả ba bộ vẫn **chưa có bằng chứng xác nhận đã tải**.

2. **Phương chạy preflight Kaggle:** ghi phiên bản Python, PyTorch, torchvision, NumPy, Pillow, GPU; import thử `cirtorch` từ checkout hiện tại và kiểm tra nạp được custom Caffe VGG16 weights.
3. **Codex tạo patch C1 có giới hạn:** đường dẫn input, weights, output và device truyền qua arguments/config, không hard-code Kaggle layout giả định. Không bắt buộc chạy download nếu file đã tồn tại; ảnh đọc từ `/kaggle/input/...`, log/checkpoint ghi `/kaggle/working/...`.
4. **Smoke test C1:** load một ảnh SfM, một mẫu query benchmark và một metadata file; load model lên device, tạo một descriptor, kiểm tra shape và toàn bộ input/output path thực tế.

Không được thay đổi: VGG16/GeM/L2, image transforms, mining, positive/negative tuples, loss hay thuật toán đánh giá. Các giá trị max image size cho training và evaluation là hai config độc lập.

Đầu ra bàn giao: `dataset_manifest.md` của Hoàng; Kaggle environment record, cấu hình C1, PR kèm diff và log load data/weights/device của Phương. **Tiêu chí hoàn thành:** không download ngoài ý muốn, không ghi vào input read-only, load được dữ liệu/weights thực và sinh được descriptor; lỗi phải có thông báo rõ.

3.4 C2 — Checkpoint, Resume & Learning-rate Continuity

C2 giải quyết vấn đề gì? Phiên Kaggle GPU có thể ngắt trước khi huấn luyện xong. Nếu checkpoint chỉ lưu model hoặc scheduler bị khởi tạo khác, lần resume có thể âm thầm làm thay đổi đường học và khiến B0 không còn tái lập được.

Các từ cần hiểu: *checkpoint* là bản lưu trạng thái training; *optimizer state* bao gồm lịch sử cập nhật tham số (ví dụ momentum); *scheduler* điều khiển learning rate (LR) theo epoch/step; *resume* là tiếp tục từ checkpoint, **không phải khởi động training mới bằng weights cũ**.

Việc cần làm theo thứ tự:

1. Nguyên đối chiếu hành vi scheduler trong paper/source với PyTorch mới và ghi quyết định LR policy của nhóm trước khi sửa. Không tự sửa gamma hoặc thời điểm `scheduler.step()` theo cảm tính.
2. Phương xác định checkpoint gồm epoch/step, model state, optimizer, scheduler, cấu hình, seed và khi cần cả RNG/sampling state; chọn thời điểm lưu nhất quán và nơi ghi có thể khôi phục giữa các phiên Kaggle.
3. Codex chỉ bổ sung save/load và logic resume; không thay optimizer, loss hoặc chu kỳ mining. File checkpoint lớn nằm ngoài Git.
4. Test một run ngắn: train một vài bước → save → tắt/recreate process → load → tiếp tục; kiểm tra LR, epoch, optimizer state và loss vẫn hợp lệ. Nếu cần kiểm tra tái lập chính xác, so sánh chuỗi không ngắt với chuỗi có resume ở cùng seed và data order.

Đầu ra bàn giao: mô tả LR policy do Phương–Nguyên duyệt; patch C2; checkpoint mẫu và log resume. **Tiêu chí hoàn thành:** resume đúng epoch/step và LR dự kiến; không lặp epoch hoặc decay LR thêm ngoài ý muốn. Chưa có bằng chứng đạt tiêu chí này.

3.5 C3 — Training Loop Stability & Sampling Guards

C3 giải quyết vấn đề gì? Một pipeline có thể import được nhưng không train đúng: tuple thiếu negatives, subset ít cluster làm mining thất bại, batch rỗng, tensor khác device, ảnh hỏng, gradient `NaN` hoặc model không được cập nhật.

Các từ cần hiểu: *query* là ảnh neo; *positive* là ảnh cùng instance có quan hệ hình học phù hợp; *negative* là ảnh khác instance; *hard-negative mining* chọn negative dễ nhầm để huấn luyện; *smoke test* là chạy rất ngắn nhằm phát hiện lỗi trước full training.

Việc cần làm theo thứ tự:

1. Hoàng xác nhận metadata và số cluster phù hợp với kích thước sampling/mining đã chọn; không chọn subset ảnh ngẫu nhiên làm sai cấu trúc SfM cluster.
2. Nguyên truy dấu trong source cách sinh tuples, số negatives, hard-negative refresh, contrastive loss, normalization và optimizer; ghi các khác biệt paper/source để nhóm quyết định B0 policy.
3. Phương/Codex thêm kiểm tra điều kiện tiên quyết và thông báo lỗi có thể hiểu được (không âm thầm đổi margin, `drop_last`, cách chọn negative hay công thức loss).
4. Chạy một mini-run: kiểm tra tensor shapes/devices, loss hữu hạn, backward chạy được, gradient không bất thường và weights thực sự cập nhật; log một vài bước. Nếu gặp OOM, ghi lại cấu hình và phương án điều chỉnh **có phê duyệt**, không sửa ngầm benchmark.

Đầu ra bàn giao: training smoke-test log, danh sách sampling guards, PR C3 và bảng những lựa chọn B0 còn cần quyết định. **Tiêu chí hoàn thành:** một mini-run đi trọn pipeline sample → forward → loss → backward → optimizer step mà không đổi methodology ngoài quyết định đã ghi.

3.6 C4 — Retrieval Evaluation & mAP Validation

C4 giải quyết vấn đề gì? Model chạy và loss giảm **chưa có nghĩa retrieval chính xác**. Một lỗi crop query, sắp thứ tự image IDs hoặc loại bỏ `junk` có thể tạo ra số mAP sai, khiến so sánh B0/B1 không đáng tin.

Các từ cần hiểu: *gallery* là toàn bộ ảnh để tìm; *query bounding box* là vùng truy vấn cần crop theo protocol; *rank* là thứ tự ảnh từ giống đến khác; *AP* đánh giá chất lượng ranking cho một query; *mAP* là trung bình AP trên nhiều query; *junk* là ảnh không được dùng như positive/negative khi tính AP trong protocol legacy.

Việc cần làm theo thứ tự:

1. Hoàng xác nhận Oxford5k/Paris6k có các file ảnh, danh sách query, bounding boxes và annotations cần cho evaluation; thống nhất legacy Oxford/Paris hay ROxford/RParis, **không trộn protocol**.
2. Nguyên đọc code evaluation, kiểm tra thứ tự descriptor/index, query crop, phép tính ranking, vị trí positive/junk, cách loại junk và công thức AP/mAP. Có thể viết unit test AP thủ công trên một ranking toy nhỏ với expected result.
3. Phương tích hợp đường dẫn benchmark và test xuất descriptors/rankings ở max evaluation image size đã quyết định. Thiết lập B0 đang định hướng **train max 362, eval max 1024**; đây là hai biến độc lập, chưa phải thông số đã nghiệm thu runtime.
4. Chạy evaluation trên model/checkpoint rõ ràng; lưu mAP theo dataset và protocol, cấu hình resize/crop, commit hash và log; nếu so sánh paper, nêu rõ khác biệt training budget, optimizer/mining, weights, whitening/QE và evaluation settings.

Đầu ra bàn giao: C4 evaluation checklist, toy AP/mAP test, bảng kết quả benchmark có đầy đủ cấu hình và PR liên quan. **Tiêu chí hoàn thành:** protocol minh bạch, AP/mAP khớp các ví dụ kiểm tra, evaluation có thể chạy lại từ cùng checkpoint; **chưa có nghĩa mAP đạt giá trị paper**.

3.7 Cổng kiểm tra trước full training

- C0 preflight/import chạy trên Kaggle.
- Dataset manifest và sample loading hợp lệ.
- C1–C4 có commit/PR và review; hoàn thành tiêu chí riêng từng giai đoạn, smoke test pass.
- Checkpoint save/resume test được, LR đúng policy đã chốt.
- Config B0, seed, dataset subset, pretrained Caffe weights và output path được ghi lại.

4. GitHub:

GitHub Organization: https://github.com/FUAVX ; repository dự kiến: https://github.com/FUAVX/IIR-Robust-Retrieval. Việc origin đã được cấu hình không chứng minh repo đã push thành công; cần kiểm tra remote branches.

Branch	Ý nghĩa	Quy tắc
main	Source GeM gốc nhập về	Giữ ổn định; không phát triển trực tiếp.
B0	Nhánh tích hợp baseline reproduction	Chỉ merge code đã review/test; ghi rõ compatibility vs methodology.
B0/kaggle-runtime	Phương: runtime / C1–C4	Tạo từ B0, PR vào B0.
B0/methodology-audit	Nguyên: docs/analysis	Tạo từ B0, PR vào B0.
B0/data-preparation	Hoàng: manifests/scripts	Tạo từ B0, PR vào B0.
B1/...	Nhánh cải tiến tương lai	Chưa tạo cho đến khi có quyết định B1.

4.1 Lần đầu tạo branch công việc

```
git fetch origin
git switch B0
git pull --ff-only origin B0
git switch -c B0/kaggle-runtime

# Làm việc, sau đó:
git status
git add path/to/changed-file
git commit -m "fix: prepare Kaggle runtime paths"
git push -u origin B0/kaggle-runtime
```

Nguyên/Hoàng thay tên branch tương ứng. Thay `path/to/changed-file` bằng đường dẫn file thật. Không chạy git add . nếu chưa xem status và .gitignore.

4.2 Pull request & review

1. Tạo branch từ B0 mới nhất; commit thay đổi nhỏ, mô tả phạm vi rõ ràng.
2. Push branch lên FUAVX và mở Pull Request với base = B0, compare = branch của mình.
3. PR nêu mục tiêu, file sửa, cách kiểm tra, kết quả và rủi ro; một thành viên khác review.

4. Nếu sửa liên quan methodology, cả nhóm xác nhận trước khi merge. Sau merge, cập nhật local B0.
5. Không force-push B0/main, không commit dataset, checkpoints, pretrained weights, secrets hoặc Kaggle credentials.

4.3 Tránh conflict thực tế

Phương ưu tiên scripts/notebooks/configs runtime; Nguyên ưu tiên report/methodology và audit; Hoàng ưu tiên manifests/data scripts. File chung như README hoặc train.py cần thông báo trước khi sửa. Nếu hai người cùng cần sửa một file, thống nhất người tích hợp và tách PR theo thứ tự.

4.4 Mẫu nội dung Pull Request

```
Title: [B0/C1] Make Kaggle paths configurable
Purpose: ...
Files changed: ...
What is preserved: GeM / loss / mining / evaluation protocol
Validation: syntax / imports / smoke test (state actual result)
Open questions: ...
Reviewer: ...
```

5. Kế hoạch nghiên cứu robustness sau B0 (định hướng, chưa triển khai)

Khi B0 đã chạy và có mAP clean, nhóm tạo một giao thức stress test query với blur, noise, ánh sáng, compression, occlusion, scale; viewpoint thật phải đánh giá bằng ảnh thực, không xem perspective warp là thay thế hoàn toàn. Đo mAP theo loại/severity và mức suy giảm so với clean; lưu các trường hợp top-ranked sai. Không dùng test set để điều chỉnh hyperparameter.

Sau failure analysis mới chọn B1, có thể thuộc nhóm robust training, adaptive pooling, multi-scale hoặc mining. Mỗi lựa chọn phải có giả thuyết, phép đối chứng, ablation và đánh giá trade-off robustness–discriminability–efficiency. Không tuyên bố novelty trước literature review.

6. Checklist họp nhóm đầu tiên

- ☐ Cả ba đọc và giải thích được pipeline query → descriptor → ranking → mAP.
- ☐ Phương xác nhận GitHub push và Kaggle preflight.
- ☐ Hoàng báo tình trạng tải/mount của cả ba dataset, file nào còn thiếu.
- ☐ Nguyên chốt danh sách methodology gaps cần quyết định trước training.
- ☐ Thống nhất deadline vòng đầu và kênh cập nhật công việc.
- ☐ Chỉ giao Codex C1 khi đã có đường dẫn dataset thật.

7. Tài liệu tham khảo và giới hạn

Paper: Fine-tuning CNN Image Retrieval with No Human Annotation — https://arxiv.org/abs/1711.02512

Source gốc: https://github.com/filipradenovic/cnnimageretrieval-pytorch

Team repo: https://github.com/FUAVX/IIR-Robust-Retrieval (cần xác nhận quyền truy cập và push).

Giới hạn: tài liệu dựa trên thông tin tiến độ và diff Codex do nhóm cung cấp; chưa trực tiếp chạy source hoặc xác minh datasets trên Kaggle. Các giá trị cấu hình đang thảo luận (chẳng hạn số epoch, subset, optimizer) chưa phải quyết định cuối cùng.